

UI AND EXECUTION TRACE

I built the CLI using a `Java Scanner` and a `switch` statement inside a `while(true)` loop to handle the mandatory simulator commands cleanly. This approach makes it easy to add more commands later.

To track the execution sequence without cluttering the main CPU logic, I created a dedicated `ExecutionTrace` class. It uses a `StringBuilder` to efficiently record and format the `FETCH`, `DECODE`, and `EXECUTE` stages before the UI prints them to the screen.

Problems Encountered & Solutions

Infinite Loops: I realized the `RUN` command could lock up the console if a loaded program was missing a termination instruction. I solved this by adding a safety counter that automatically breaks the loop after 1,000 iterations.

Integration Delays: My UI code relies on backend methods (like fetching the CPU state or status) that the rest of the team was still writing. We bypassed this by agreeing on exact method names early on, allowing me to code against empty "dummy" methods until the actual logic was merged.